- A form feed, \f or unicode \x0c
- A vertical tab, \v or unicode \x0b
- Unicode null, x00 or \u0000

Removing the Last Character from a String

When doing text processing, especially from files or command-line input, a common task is to remove the last character of a string if it is the end of line marker. Ruby adapted a couple of versions of this feature from Perl.

The method chop always removes the last character of a string, but if the last two characters of the string are \r\n (the Windows end of line marker), chop removes both of them.

The similarly named but more useful in practice method called chomp removes the last character if it is a line separator. By default that means \r, \n, or \r\n but not—the documentation clearly points out—\n\r.

You can pass an argument to chomp. If that argument is an empty string, chomp will remove a series of \n or \r\n characters at the end of the string. If it's any other character, chomp will remove that character once if it is the last character of the string.

All these versions with arguments seem likely to be confusing. As such, we recommend using chomp without arguments if you need this functionality and perhaps writing a more explicit version of anything more complex that you need. Both chop and chomp have modifier versions, chop! and chomp!. As with other versions we've seen, they return the new string if the string changed and nil otherwise.

Iterating within Strings

If you want to iterate over different parts of the string, you have a few options that allow you to split a string up, or options to perform the iteration without actually splitting the string.

The method chars splits the string up into an array of characters. Ruby doesn't actually have a character class, so chars splits the string up into an array of one-length strings. The method each_char takes a block and calls the block with each character in succession. As with other enumeration methods, if called without a block, the method returns a Ruby enumerator that can be invoked later or chained to other enumerators.

Similar methods break the string down a little differently. The method bytes returns an array of the numerical bytes that make up each character, where some characters might be more than one byte long. The codepoints method returns the codepoint for each character in the current encoding. That array produces one entry for each character. The iterators each_byte and each_codepoint take blocks or return enumerators and allow you to enumerate over the bytes or code points.

The method lines splits the string into an array of strings based on line separators. By default, the separator is $/, the global line separator, but you can pass your own line separator as any character you want. If the argument is an empty string, lines(''), then the method acts as a paragraph splitter and splits when there are two or more line separators in a row. An optional second argument, chomp: true, removes the last instance of the line separator from each element in the array. The related method each_lines takes the same argument and returns an enumerator, or takes a block argument and invokes the block ones for each separated line.